Testausdokumentti

Harjoitustyö: Connect 4 -tekoäly

1. Mitä on testattu, miten tämä tehtiin?

Tällä hetkellä on tehty yksinkertaisia yksikkötestejä, siinä mielessä että testien tekeminen onnistuisi. Suurin osa ajasta on mennyt algoritmin kirjoittamiseen. Testit ovat python unittestejä.

Testi 1, yksittäinen siirto:
Testi antaa ohjelmalle `MOVE:0`-komennon ja tarkistaa, että ohjelma palauttaa siirron sarakkeeseen 0.

Testi 2, ensimmäinen siirto:
Testi antaa ohjelmalle `PLAY:`-komennon ja tarkistaa, että tekoäly valitsee ensimmäiseksi siirroksi keskimmäisen sarakkeen eli sarakkeen 3.

Testi 3, laudan täyttö:
Testi pelaa 42 siirtoa ja tarkistaa, että jokaiselle siirrolle saadaan kelvollinen sarake väliltä 0–6. Koska Connect 4 -laudalla on 42 paikkaa, tämä testaa myös sitä, että tekoäly pystyy pelaamaan koko laudan täyteen.

Testi 4, laudan ylitäyttö:
Testi pelaa 43 siirtoa ja tarkistaa, että 43. siirron jälkeen ohjelma palauttaa `MOVE: -1`, kun laudalla ei ole enää vapaita paikkoja.

Testi 5, ylitäytön käsittely:
Testi pelaa 43 siirtoa ja tarkistaa, että ohjelma tulostaa yhteensä 43 siirtoriviä. Testi epäonnistuu esimerkiksi silloin, jos ohjelma kaatuu ennen 43. siirron käsittelyä.

Testi 6, laudan tarkistus:
Testi pelaa 42 siirtoa ja käyttää tämän jälkeen lisäämääni `CURRENT:`-komentoa. Testi tarkistaa, että komento tulostaa laudan nykyisen tilan oikeassa muodossa.

Testi 7, välitön pystysuora voitto:
Testi muodostaa laudalle tilanteen, jossa pelaajalla on kolme omaa nappulaa peräkkäin pystysuunnassa. Tämän jälkeen tekoälylle annetaan `PLAY:`-komento ja tarkistetaan, että se tunnistaa voittavan siirron sarakkeessa 3.

Testi 8, välitön vaakasuora voitto:
Testi muodostaa laudalle tilanteen, jossa pelaajalla on kolme omaa nappulaa peräkkäin vaakasuunnassa. Tämän jälkeen tekoälylle annetaan `PLAY:`-komento ja tarkistetaan, että se tunnistaa voittavan siirron seuraavaan tarvittavaan sarakkeeseen.

Testi 9, välitön diagonaalivoitto:
Testi muodostaa laudalle tilanteen, jossa pelaajalla on kolme omaa nappulaa peräkkäin vinottain. Tämän jälkeen tekoälylle annetaan PLAY:-komento ja tarkistetaan, että se tunnistaa voittavan siirron sarakkeeseen 3.
Testi 10, välittömän pystysuoran voiton estäminen:

Testi muodostaa laudalle tilanteen, jossa vastustajalla on kolme omaa nappulaa peräkkäin pystysuunnassa. Tämän jälkeen tekoälylle annetaan PLAY:-komento ja tarkistetaan, että se estää vastustajan voiton pelaamalla sarakkeeseen 3.

Testi 11, välittömän vaakasuoran voiton estäminen:
Testi muodostaa laudalle tilanteen, jossa vastustajalla on kolme omaa nappulaa peräkkäin vaakasuunnassa. Tämän jälkeen tekoälylle annetaan PLAY:-komento ja tarkistetaan, että se estää vastustajan voiton pelaamalla seuraavaan tarvittavaan sarakkeeseen.

Testi 12, välittömän diagonaalivoiton estäminen:
Testi muodostaa laudalle tilanteen, jossa vastustaja uhkaa diagonaalivoittoa. Tämän jälkeen tekoälylle annetaan PLAY:-komento ja tarkistetaan, että se estää vastustajan voiton pelaamalla sarakkeeseen 4.

Testi 13, ensimmäinen siirto keskelle:
Testi antaa ohjelmalle PLAY:-komennon tyhjällä laudalla ja tarkistaa, että tekoäly valitsee sarakkeen 3.

Testi 14, keskustan suosiminen:
Testi muodostaa tilanteita, joissa mikään siirto ei ole pakotettu ja reunasarakkeet ovat houkuttelevia. Testi tarkistaa, että tekoäly suosii tällöin keskimmäistä saraketta 3.

Testi 15, yhdistetyt uhat:
Testi muodostaa laudalle tilanteen, jossa pelaajalla on nappulat sarakkeissa 2 ja 4. Testi tarkistaa, että tekoäly pelaa sarakkeen 3, jolloin syntyy avoin kolmonen eli kaksi uhkaa kerralla yhden uhan sijaan.

Testi 16, pelattavuuden huomiointi:
Testi tarkistaa, että tekoäly huomioi laidat, yläreunan ja vastustajan nappulat uhkien luomisessa. Pelkillä reunapaloilla tekoäly suosii avointa keskustaa, eikä se pelaa reunasaraketta 0 tai 6, jos vastustaja blokkaa sarakkeen 3 eikä neljän suoraa voi muodostaa reunaan päin.

Testi 17, yhtenäisten nappuloiden suosiminen:
Testi muodostaa tilanteen, jossa tekoälyllä on nappula keskellä lautaa. Testi tarkistaa, että tekoäly rakentaa nappulan viereen eikä kauas siitä.

Testit suoritetaan Pythonin `unittest`-kirjastolla. Ohjelmaa ajetaan testeissä erillisenä prosessina `subprocess`-kirjaston avulla, jolloin testeille voidaan antaa samoja komentoja kuin ohjelman normaalissa käytössä.

2. Miten testit voidaan toistaa?

Unittestaus:

Avaa terminaalissa projektikansio:

Varmista että poetry on alustettuna ja riippuvuudet ajan tasalla.

Sitten voit ajaa: poetry run python -m unittest tests.test_connect4 -v

3. Ohjelman toiminnan mahdollisen empiirisen testauksen tulosten esittäminen graafisessa muodossa.

Ei valmista.